

Reviewer Pack — Lattice-Valued Topological Entropy Lower Bound for Boolean C...

Entry 0d89b3d099a5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 16:32:51 UTC

Lattice-Valued Topological Entropy Lower Bound for Boolean Circuit Depth

Entry ID: 0d89b3d099a5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 16:32:51 UTC

1. Conjecture

Field A (mathematical branch): Lattice Theory (specifically, lattice-valued topology) **Field B** (complexity object): Boolean circuit complexity (circuit depth)

Statement:

For every boolean circuit C with n inputs and depth D , the topological entropy of the lattice of its output values is lower bounded by a function $\alpha(D)$ such that $\alpha(D) = \Omega(D)$. Specifically, there exists a constant $c > 0$ such that for all circuits C , $H(C) \geq c * \log_2(n + D)$, where $H(C)$ denotes the topological entropy of C .

Rationale (proposer's reasoning):

Lattice-valued topology provides a framework to study the complexity of functions in terms of their algebraic structure. The conjecture links this field with boolean circuit complexity by proposing that the complexity of a circuit can be characterized by the topological entropy of its output values' lattice. This may reveal deeper connections between algebraic and computational complexity.

Taxonomy category: lattice_entropy_lower_bound (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8964712f1a534395

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all circuit depths $D \leq \log_2(n + D)$, the empirical mean of topological entropies $H(C)$ across n input sizes $\{2, 4, 8\}$ satisfies $H(C)_{\text{mean}} \geq c * \log_2(n + D)$ with a constant $c > 0$. The conjecture is falsified if any generated circuit with depth $D \leq \log_2(n + D)$ has a topological entropy $H(C) < c * \log_2(n + D)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - lattice-valued topology AND boolean circuit depth - topological entropy lower bound lattice theory circuit complexity - Boolean circuit depth and lattice-valued topological entropy

Top relevant hits considered: - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/1612.04878v1] Free Boolean Topological Groups - [http://arxiv.org/abs/1703.08540v1] Depth-Optimal Quantum Circuit Placement for Arbitrary Topologies - [http://arxiv.org/abs/0710.4339v1] Heavy-Quark Masses from the Fermilab Method in Three-Flavor Lattice QCD - [http://arxiv.org/abs/1111.0981v1] Form factors for B to $K\ell$ semileptonic decay from three-flavor lattice QCD - [http://arxiv.org/abs/1110.6876v4] Lower bounds for topological complexity - [http://arxiv.org/abs/1712.05384v2] Simulation of low-depth quantum circuits as complex undirected graphical models - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from math import log2, ceil

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [2, 4, 8]
    total_entropy = 0
    instances_tested = 0
    n_max = 0

    for n in n_values:
        D = ceil(log2(n + 1))
        if D == 0: continue

        # Generate all possible boolean circuits with depth D
        def generate_circuits(depth, inputs):
            if depth == 0:
                return [inputs]
            circuits = []
            for i in range(1 <= len(inputs)):

```

```

        circuit = []
        for j in range(len(inputs)):
            if (i >> j) & 1:
                circuit.append('NOT ' + inputs[j])
            else:
                circuit.append(inputs[j])
        circuits.extend(generate_circuits(depth - 1, circuit))
    return circuits

circuits = generate_circuits(D, [f'x{i}' for i in range(n)])

for circuit in circuits:
    # Calculate the output lattice of the circuit
    def evaluate(circuit):
        if 'NOT ' in circuit:
            subcircuit = circuit.replace('NOT ', '')
            return {1 - evaluate(subcircuit)}
        else:
            inputs = [int(x) for x in circuit.split()]
            return set(inputs)

    output_lattice = evaluate(circuit)
    entropy = len(output_lattice) * log2(len(output_lattice))
    total_entropy += entropy
    instances_tested += 1
    n_max = max(n_max, n)

mean_entropy = total_entropy / instances_tested if instances_tested > 0 else 0

c = 0.5 # Example constant for the lower bound
conjecture_holds = mean_entropy >= c * log2(n + D) for n in n_values and D == ceil(log2(n + 1))

counterexample = "" if conjecture_holds else "mean_entropy < c * log2(n + D)"

return {
    "metric_name": "topological entropy",
    "metric_value": mean_entropy,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

)}}
TRIAL: {'metric_name': 'topological entropy', 'metric_value': -2.0, 'instances_tested': 3, 'n_max': 8}
TRIAL: {'metric_name': 'topological entropy', 'metric_value': -2.0, 'instances_tested': 3, 'n_max': 8}
TRIAL: {'metric_name': 'topological entropy', 'metric_value': -2.0, 'instances_tested': 3, 'n_max': 8}
TRIAL: {'metric_name': 'topological entropy', 'metric_value': -2.0, 'instances_tested': 3, 'n_max': 8}
TRIAL: {'metric_name': 'topological entropy', 'metric_value': -2.0, 'instances_tested': 3, 'n_max': 8}
TRIAL: {'metric_name': 'topological entropy', 'metric_value': -2.0, 'instances_tested': 3, 'n_max': 8}
TRIAL: {'metric_name': 'topological entropy', 'metric_value': -2.0, 'instances_tested': 3, 'n_max': 8}
TRIAL: {'metric_name': 'topological entropy', 'metric_value': -2.0, 'instances_tested': 3, 'n_max': 8}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6a12b7d3.py", line 105, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6a12b7d3.py", line 105, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Re-run the test to ensure it completes without crashing and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	29963
2	preregistration	ollama_remote	glm4:latest	0	0	28723
3	novelty	ollama_remote	glm4:latest	0	0	14344
4	novelty	ollama_remote	glm4:latest	0	0	16883
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13819
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16721
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13727
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14113
9	judge	ollama_remote	glm4:latest	0	0	20789

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 169082 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0d89b3d099a5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0d89b3d099a5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0d89b3d099a5.tar.gz` (if generated)